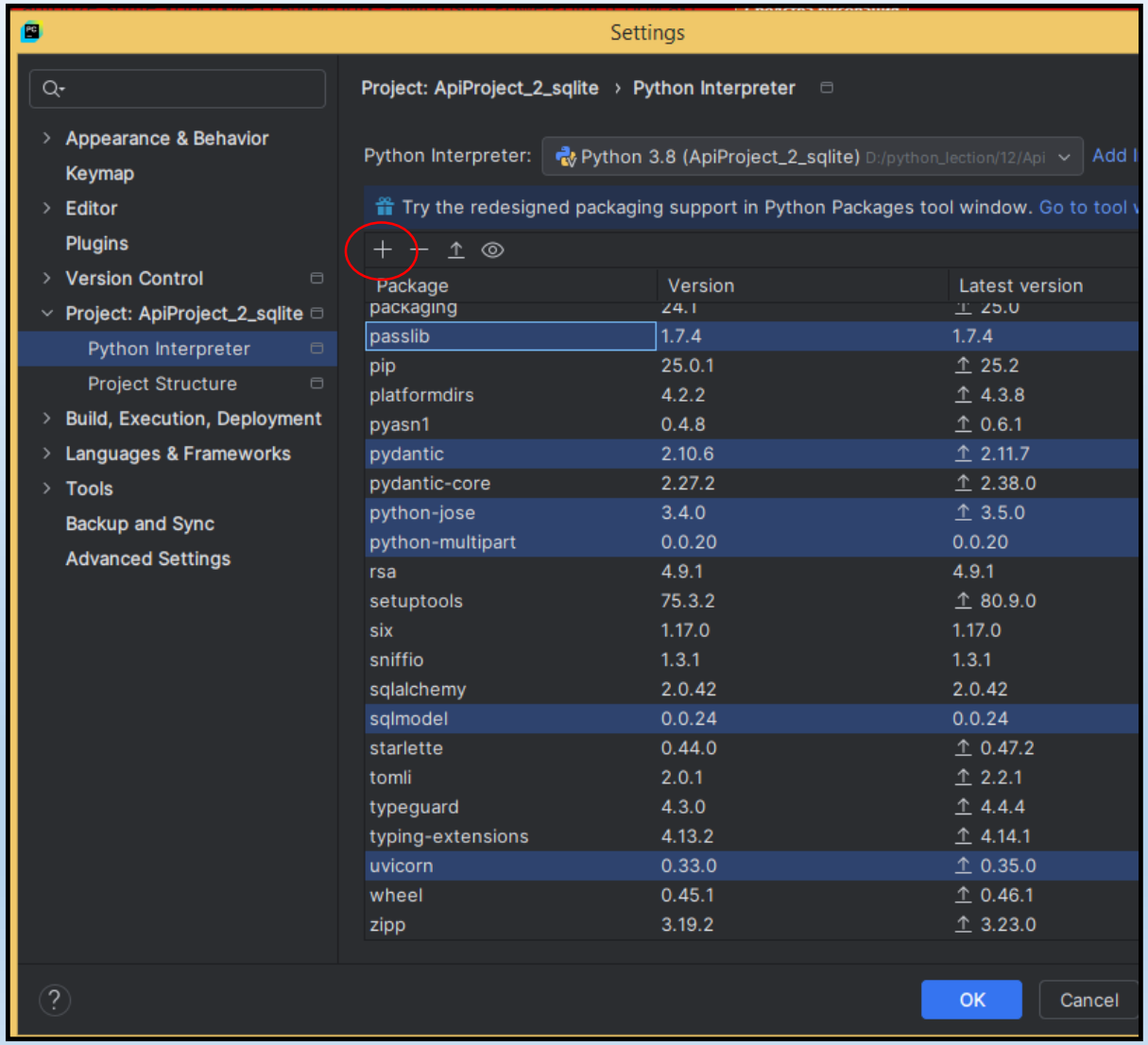
Создание API



Подключение библиотек

uvicorn
fastapi
sqlmodel
email-validator
asyncpg
alembic
psycopg2
python-dotenv
python-jose
python-multipart



Установка библиотек

pip install python-dotenv psycopg2 sqlmodel uvicorn fastapi

Установка с локального репозитория

pip install --no-index --find-links=\\172.30.44.161\public python-dotenv
pip install --no-index --find-links=\\172.30.44.161\public psycopg2
pip install --no-index --find-links=\\172.30.44.161\public sqlmodel
pip install --no-index --find-links=\\172.30.44.161\public fastapi

Библиотека fastapi

fastapi - высокопроизводительный веб-фреймворк для создания API на Python 3.6+

- Встроенная валидация и сериализация данных с помощью Pydantic
- Поддержка асинхронного программирования
- Автоматическая генерация документации OpenAPI и Swagger UI
- Удобная работа с параметрами пути, запросов и тел запроса
- Обработка событий сервера (startup, shutdown)
- Интуитивный синтаксис с использованием декораторов для маршрутизации

Библиотека sqlmodel

sqlmodel - ORM на основе Pydantic и SQLAlchemy

- Позволяет описывать модели данных с валидацией и типами, поддерживая работу с базами данных
- Интеграция с FastAPI для простого создания CRUD-приложений
- Поддерживает миграции и асинхронные операции с базой данных

Библиотека pydantic

pydantic - Библиотека для валидации данных с использованием Python-type hints

- Создание моделей с строгим типом данных и автоматической проверкой
- Поддержка сложных вложенных структур данных, сериализация/десериализация
- Основной строительный блок валидации данных в FastAPI и SQLModel
- Удобный и мощный инструмент для обработки пользовательского ввода и конфигураций

Библиотека python-multipart

python-multipart - для обработки HTTP-запросов с типом содержимого multipart/form-data при загрузке файлов через веб-формы, эндпоинты (для Api).

Позволяет:

- корректно разбирать входящие данные, разделённые на несколько частей (например, поля формы и файлы)
- Обеспечение корректного получения содержимого из запроса без необходимости парсить данные вручную.

Библиотека psycopg2

psycopg2 —адаптер для работы с базой данных PostgreSQL из Python. Она позволяет Python-приложениям устанавливать соединение с PostgreSQL, выполнять SQL-запросы, управлять транзакциями и работать с различными типами данных PostgreSQL напрямую из кода на Python.

Библиотека python-dotenv

python-dotenv -Библиотека предназначена для работы с переменными окружения в Python-проектах.

Считывает пары «ключ=значение» из файла .env и загружает эти переменные в окружение процесса, чтобы ваш код мог получить к ним доступ через стандартные средства Python (os.getenv, os.environ).

Структура папок

```
name_project/
├── app/
│   ├── main.py                # Точка входа
│   ├── api/                   # Маршруты
│   │   ├── v1/
│   │   │   ├── api_router_имя_модели.py
│   ├── models/                # Модели (Model)
│   │   ├── __init__.py
│   │   ├── имя_класса.py
│   ├── controllers/           # Контроллеры (Controller) - бизнес-логика
│   │   ├── имя_класса_controller.py
│   ├── db/                    # Настройка базы данных
│   │   ├── database.py
│   │   ├── session.py
│   ├── schemas/               # схемы Pedantic
│   │   ├── имя_класса_schema.py
│   ├── core/                  # Общие утилиты и конфигурации
│   │   ├── __init__.py
│   │   ├── config.py          # Конфигурация, переменные окружения
│   │   ├── swagger_ui.py      # Функции get_swagger_ui_html и get_swagger_ui_oauth2_redirect_html библиотеки fastapi
│   │   ├── security.py        # Функции безопасности и auth helpers
│   │   ├── utils.py           # Другие вспомогательные функции
├── .env                       # Переменные окружения
├── .gitignore
├── requirements.txt           # Зависимости
└── README.md                  # Документация проекта
```

Управление параметрами конфигурации

Установите python-dotenv

```
from dotenv import load_dotenv
import os

load_dotenv()  # загрузка переменных из .env

secret_key = os.getenv("SECRET_KEY")
database_url = os.getenv("DATABASE_URL")
```

```
DATABASE_URL="postgresql+psycopg2://DB_USER:DB_PASSWORD@DB_HOST:DB_PORT/DB_NAME"
SECRET_KEY="scret_key_for_demo_new"
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

Библиотека alembic

Alembic — это инструмент для миграций базы данных в Python-приложениях (из python в PostgreSql)

Миграции — это система контроля изменений структуры базы данных (БД), которая позволяет:

- Синхронизировать модели проекта с таблицами БД.
- Отслеживать историю изменений схемы БД (аналог Git для базы данных).
- Безопасно применять/откатывать изменения в командной работе.

Инструкция по подключению Alembic

Миграция с помощью Alembic

```
my_fastapi_project/
├── alembic.ini        # Конфигурация Alembic
├── alembic/           # Папка с миграциями Alembic (или migrations/)
│   ├── env.py
│   ├── versions/
│   └── script.py.mako
├── app/               # Исходный код приложения
│   ├── main.py
│   ├── models/
│   ├── api/
│   └── ...
├── requirements.txt
├── .env
└── README.md
```

Обращение к базе данных

```
app/
├── db/
│   ├── database.py   # init_db, close_db, get_engine
│   ├── session.py    # get_session
```

Подключение к БД (файл db/database.py)

```
from sqlmodel import SQLModel
from sqlalchemy import  create_engine
from dotenv import load_dotenv
import os

load_dotenv()
secret_key = os.getenv("SECRET_KEY")
DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL, echo=True)

def init_db():
    SQLModel.metadata.create_all(engine)

def close_db():
    engine.dispose()  # Освобождение ресурсов

def get_engine():
    """Получение engine для миграций"""
    return engine
```

Подключение к БД при запуске сервера (файл main.py)

```
from fastapi import FastAPI
from contextlib import asynccontextmanager


@asynccontextmanager
async def on_startup(app: FastAPI):
    init_db()
    yield
    close_db()


app_v1 = FastAPI(lifespan=on_startup)
```

Запуск в offline-режиме (файл main.py)

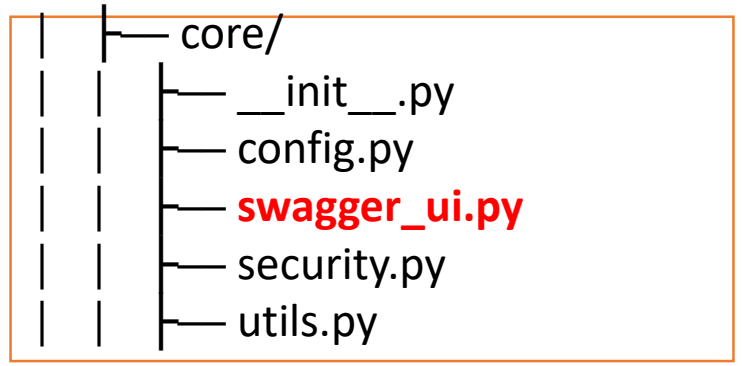
```
from contextlib import asynccontextmanager
@asynccontextmanager
async def on_startup(app: FastAPI):
    init_db()
    yield
    close_db()
app_v1 = FastAPI(
    docs_url=None,
    redoc_url=None,
    title="API v1",
    version="1.0.0",
    lifespan=on_startup)


from core import swagger_ui       # импортируем  swagger-ui
# Монтируем локальные файлы swagger-ui (предварительно поместите папку dist в “корень” проекта
app_v1.mount("/dist", StaticFiles(directory="dist"), name="dist")
```

Запуск в offline-режиме (файл swagger_ui.py)

```
from fastapi.openapi.docs import get_swagger_ui_html
from fastapi.openapi.docs import get_swagger_ui_oauth2_redirect_html
from app.main import app_v1
```

```
@app_v1.get("/api/v1/docs", include_in_schema=False)
async def custom_swagger_ui():
    return get_swagger_ui_html(
        openapi_url=app_v1.openapi_url,
        title="",
        swagger_js_url="/dist/swagger-ui-bundle.js",
        swagger_css_url="/dist/swagger-ui.css",
        oauth2_redirect_url=app_v1.swagger_ui_oauth2_redirect_url,
    )
```




```
@app_v1.get(app_v1.swagger_ui_oauth2_redirect_url, include_in_schema=False)
async def swagger_oauth2_redirect():
    return get_swagger_ui_oauth2_redirect_html()
```

ASGI серверы

Uvicorn



- ASGI-сервер для запуска FastAPI и других ASGI-приложений.
- Оптимизирован для высокой производительности.
- Поддерживает HTTP/1.1 и WebSockets.
- Лёгкий и простой в использовании.

Hypercorn



- Также ASGI-сервер, но с поддержкой HTTP/2.
- Подходит для более сложных сценариев (например, HTTP/2).
- Менее популярен для FastAPI.

Библиотека uvicorn

FastAPI не содержит встроенного сервера -> нужно установить сервер. Например, uvicorn

Без uvicorn FastAPI просто библиотека, а не сервер.

Основная команда запуска из **виртуального окружения**:

uvicorn app.main:app_v1

uvicorn app.main:app_v1 --reload

uvicorn app.main:app_v1 --port 8001

Остановка сервера: Ctrl+C



Документация API

1. http://localhost:8000/docs - Swagger UI

Удобная форма, чтобы тестировать свои запросы прямо из браузера

3. http://localhost:8000/redoc - ReDoc Красиво оформленная документация



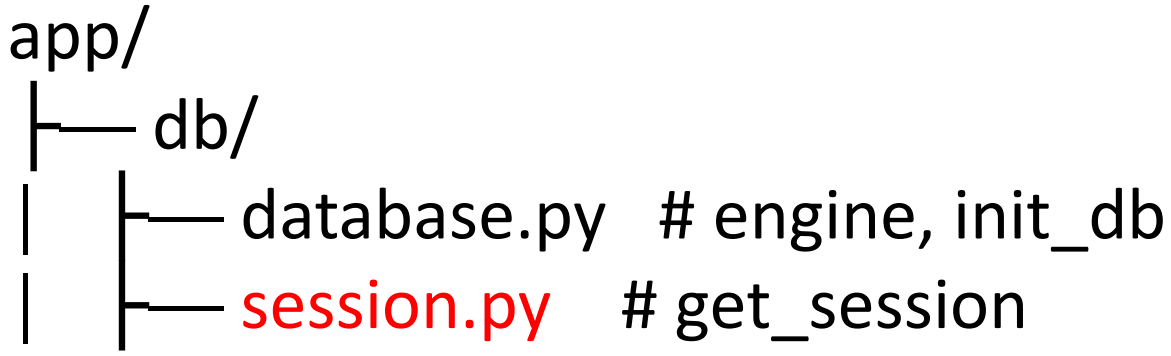


команда запуска:
uvicorn main:app --reload

Использование Depends в функциях



FastAPI


app/
db/
database.py # engine, init_db
session.py # get_session

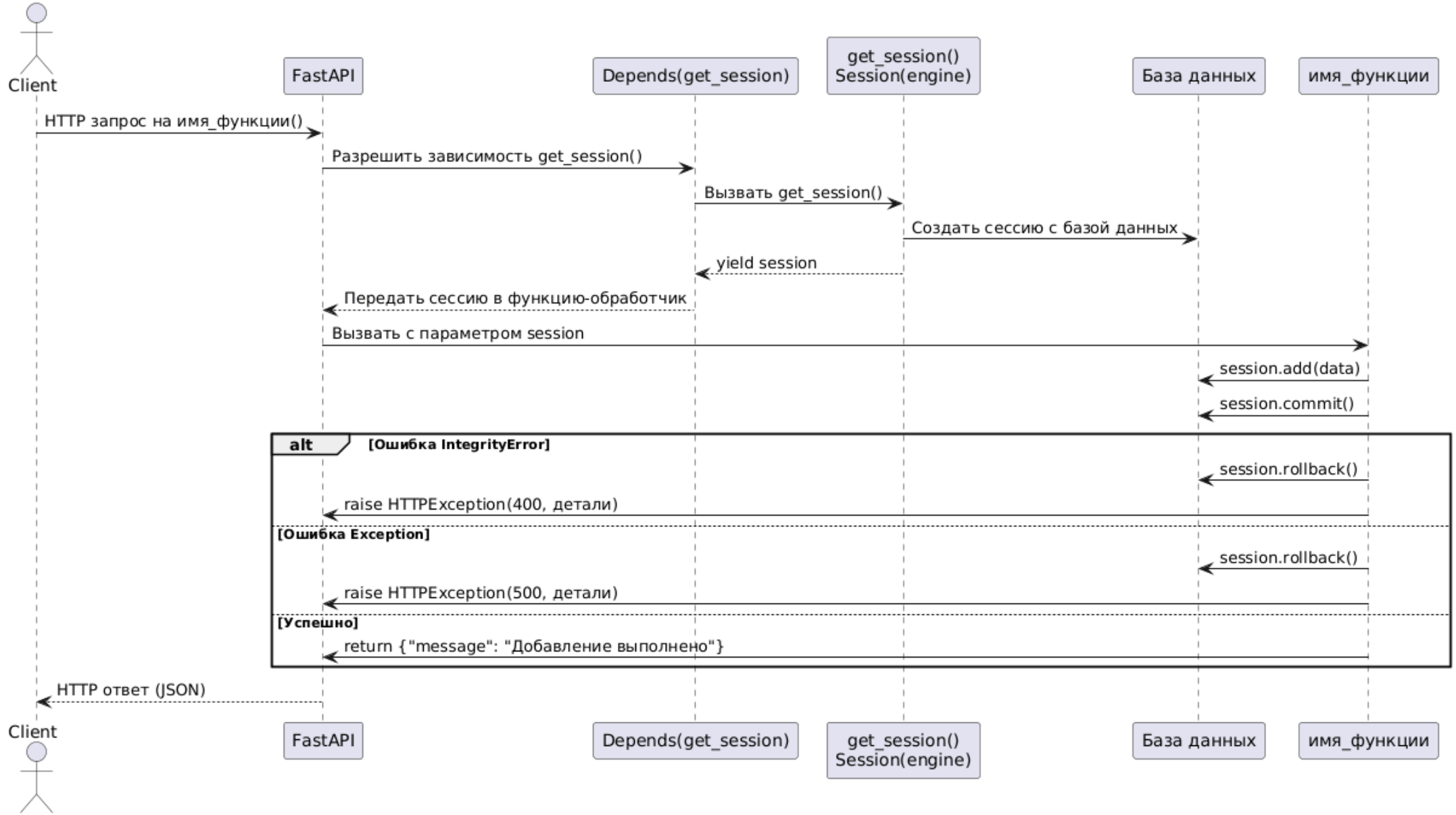
создание и управление жизненным циклом сессии SQLModel

```
from sqlmodel import  Session
from app.db.database import engine



def get_session():
    with Session(engine) as session:
        yield session
```

Что делает Depends?



Вывод всех записей из таблицы

```
from fastapi import Depends, HTTPException, status
from app.db.session import get_session
from app.models.имя_модуля import ИмяКласса


def имя_функции(session: Session = Depends(get_session)) -> List[ИмяКласса]:
    """     Вывод информации     """
    try:
        sql=select(ИмяКласса)
        result=session.exec(sql).all()
        return result
    except Exception as e:
        raise HTTPException( status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                                        detail=f"Внутренняя ошибка сервера: {str(e)}")
```

Поиск по идентификатору

```
from fastapi import Depends, HTTPException, status
from app.db.session import get_session
from app.models.имя_модуля import ИмяКласса


def имя_функции(идентификатор: int, session: Session = Depends(get_session)) -> ИмяКласса:
    """     Поиск по идентификатору    """
    try:
        result = session.get(ИмяКласса, идентификатор)
        if not result :
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"идентификатор не найден")
        return result
    except Exception as e:
        raise HTTPException( status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                                        detail=f"Внутренняя ошибка сервера: {str(e)}")
```

Добавление

```
from fastapi import Depends, HTTPException, status
from app.db.session import get_session
from sqlalchemy.exc import IntegrityError
from app.models.имя_модуля import ИмяКласса

def имя_функции(data: ИмяКласса, session: Session = Depends(get_session)) -> Optional[ИмяКласса]:
    """    Добавление    """
    try:
        session.add(data)
        session.commit()
        session.refresh(data)
        return data
    except IntegrityError:
        session.rollback()
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
                                    detail="Ошибка: дубликат или нарушение целостности данных")
    except Exception as e:
        session.rollback()
        raise HTTPException( status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                                    detail=f"Внутренняя ошибка сервера: {str(e)}")
```

Удаление

```
from fastapi import Depends, HTTPException, status, Response
from sqlmodel import Session
from app.db.session import get_session
from app.models.имя_модуля import ИмяКласса

def имя_функции(идентификатор: int, session: Session = Depends(get_session)) -> Response:
    """Удаление объекта по идентификатору"""
    try:
        result = session.get(ИмяКласса, идентификатор)
        if not result:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="идентификатор не найден")
        session.delete(result)
        session.commit()
        return Response(status_code=status.HTTP_204_NO_CONTENT)
    except Exception as e:
        session.rollback()
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Внутренняя ошибка сервера: {str(e)}"
        )
```

Редактирование

```
from fastapi import Depends, HTTPException, status
from app.db.session import get_session
from app.models.имя_модуля import ИмяКласса

def имя_функции(идентификатор: int, data: ИмяКласса , session: Session = Depends(get_session)) -> ИмяКласса:
    """    Изменение    """
    try:
        result= session.get(ИмяКласса, идентификатор)
        if not result :
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"идентификатор не найден")
        for key, value in data.dict(exclude_unset=True).items():
            setattr(result, key, value)
        session.add(result)
        session.commit()
        session.refresh(result)
        return result
    except Exception as e:
        session.rollback()
        raise HTTPException( status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                                    detail=f"Внутренняя ошибка сервера: {str(e)}")
```

Что делает Depends?

- При каждом HTTP-запросе FastAPI вызывает функцию get_session, создаёт новую сессию к базе, и передаёт эту сессию в параметр session.

- Разработчику не нужно создавать и закрывать сессию вручную — всё происходит автоматически для каждого запроса.

- Такой подход обеспечивает "чистый" код, централизует управление ресурсами и предотвращает утечки соединений или неправильное переиспользование объектов сессии при параллельных запросах.

Статус ответа при добавлении

```
router = APIRouter()

@router.post("/ИмяКласса", status_code=status.HTTP_201_CREATED)
def имя_функции(..., session: Session = Depends(get_session)) :
    """
    Описание функции
    """
    pass
```

```
    ├── api/                    # Маршруты
│   │   ├── v1/
│   │   │   ├── api_router_имя_модели.py
```

Статус ответа при удалении

```
router = APIRouter()

@router.delete("/ИмяКласса/{идентификатор}", status_code=status.HTTP_204_NO_CONTENT)
def имя_функции(идентификатор: int,  session: Session = Depends(get_session)) :
    pass
```

```
├── api/                    # Маршруты
│   ├── v1/
│   │   ├── api_router_имя_модели.py
```

Создание функции и создание запроса

```
router = APIRouter()
@router.post("/ИмяКласса",status_code=status.HTTP_201_CREATED)
def имя_функции_запроса(data: ИмяКласса, session: Session = Depends(get_session)):
    return имя_функции(data, session)


def имя_функции(data: ИмяКласса, session: Session = Depends(get_session)) :
    """    Добавление данных """
    try:
        session.add(data)
        session.commit()
        return {"message": "Добавление выполнено"}
    except IntegrityError:
        session.rollback()
        raise HTTPException(status_code=400, detail="Ошибка")
    except Exception as e:
        session.rollback()
        raise HTTPException(status_code=500, detail=f"Внутренняя ошибка сервера: {str(e)}")
```



Что такое APIRouter?

•APIRouter — это класс FastAPI, позволяющий группировать маршруты (эндпоинты) приложения.
• это "мини-приложение" для обработки запросов с собственными маршрутами, зависимостями, префиксами путей и тегами.
•разбивает большие приложения на отдельные логические части или модули.
•Позволяет создавать маршруты в отдельных файлах, чтобы структура кода была более читабельной и удобной для поддержки.

Подключение маршрутов при запуске сервера (файл main.py)

```
from fastapi import FastAPI
from contextlib import asynccontextmanager


@asynccontextmanager
async def on_startup(app: FastAPI):
    init_db()
    yield
    close_db()


app_v1 = FastAPI(lifespan=on_startup)
app_v1.include_router(router, prefix="/api/v1")
```

Версионность api



API v1 1.0.0 OAS 3.1

/api/v1/openapi.json

Url

- POST /api/v1/url/short_link Создание короткой ссылки
- GET /api/v1/url/{short_id} Просмотр информации о ссылке
- GET /api/v1/url/stats/{short_id} Get Stats By Id

User

- POST /api/v1/user_table/users/register Register User
- POST /api/v1/user_table/token Get Token
- GET /api/v1/user_table/users/ Get Users
- GET /api/v1/user_table/users/{user_id} Search User
- DELETE /api/v1/user_table/users/{user_id} Del User



API v2 2.0.0 OAS 3.1

/api/v2/openapi.json

Url

- POST /api/v2/url/short_link Создание короткой ссылки
- GET /api/v2/url/{short_id} Просмотр информации о ссылке
- GET /api/v2/url/stats/{short_id} Get Stats By Id

User

- POST /api/v2/user/users/register Register User
- POST /api/v2/user/token Get Token
- GET /api/v2/user/users/{user_id} Search User

/api/v1/docs и /api/v2/docs показывают документации нужной версии

Документация API

1. http://localhost:8000/api/v1/docs - Swagger UI

Удобная форма, чтобы тестировать свои запросы прямо из браузера

3. http://localhost:8000/api/v1/redoc - ReDoc

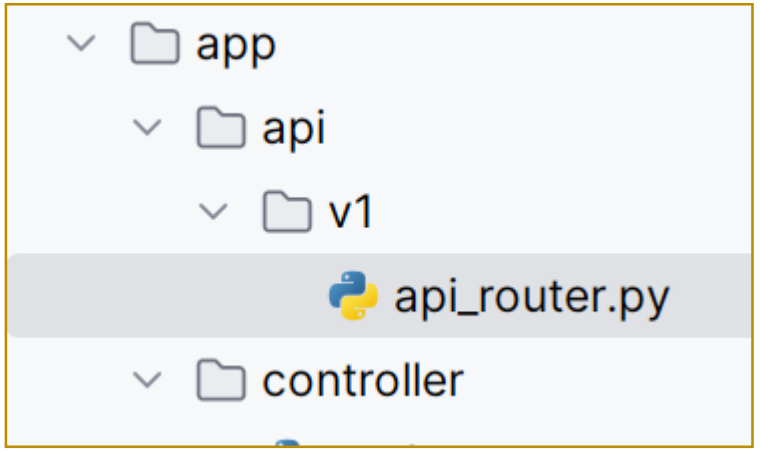
   Красиво оформленная документация





команда запуска:
uvicorn app.main:app_v1 --reload

Версионность api (файл main.py)




```
from app.api.v1 import router as псевдоним
main_app=FastAPI()
@asynccontextmanager
async def on_startup(app: FastAPI):
    init_db()
    yield
    close_db()
app_v1= FastAPI(title="API v1", version="1.0.0",
openapi_url="/api/v1/openapi.json", docs_url="/api/v1/docs",
redoc_url="/api/v1/redoc", lifespan=on_startup)

main_app.mount("/api/v1", app_v1)

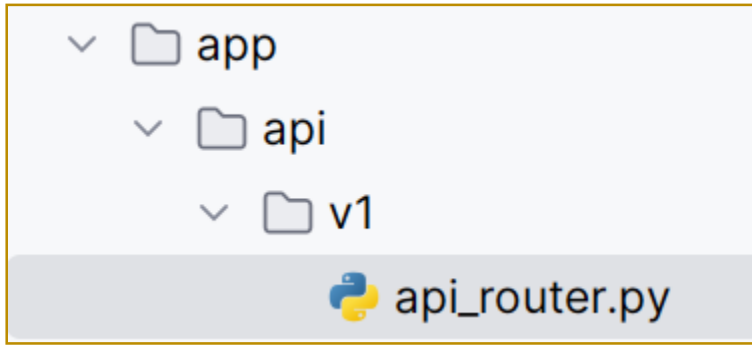
app_v1.include_router(псевдоним)
```

Создание APIRouter() – файл api_router.py

```
from fastapi import APIRouter, HTTPException, Depends
from sqlmodel importSession
from app.db.session import get_session
from app.models.имя_файла_класса import имя_класса

router = APIRouter()

@router.delete("/endpoint",description="Описание",tags=["имя_класса"])
def del_router(параметры, session: Session = Depends(get_session)):
    return имя_функции(параметры, session)
```



Файл main

```
├── app/
│   ├── main.py # Точка входа
│   ├── api/ # маршруты
│   │   ├── v1
│   │   │   ├── api_router_catalog.py
│   │   │   ├── api_router_materials.py
│   │   │   ├── api_router_details.py
│   │   │   ├── api_router_colors.py
│   │   ├── v2
│   │   │   ├── api_router_catalog.py
│   │   │   ├── api_router_materials.py
│   │   │   ├── api_router_details.py
│   │   │   ├── api_router_colors.py
```

Запуск приложения
uvicorn app.main:app_v2

Запуск приложения
uvicorn app.main:app_v1

```
from app.api.v1.имя_файла    import router as псевдоним_1
from app.api.v2.имя_файла    import router as псевдоним_2
main_app = FastAPI()

@asynccontextmanager
async def on_startup(app: FastAPI):
    init_db()
    yield
    close_db()
app_v1 = FastAPI(    title="Catalog API v1",
version="1.0.0",    openapi_url="/openapi.json",
docs_url="/docs", redoc_url="/redoc" ,    lifespan=on_startup,)

app_v2 = FastAPI(     title="Catalog API v2",
version="2.0.0", openapi_url="/openapi.json",
docs_url="/docs",       redoc_url="/redoc" , lifespan=on_startup,)

main_app.mount("/api/v1", app_v1)
main_app.mount("/api/v2", app_v2)

app_v1.include_router(псевдоним_1)
app_v2.include_router(псевдоним_2)
```

Пагинация и метаданные ответов



FastAPI

Подключение библиотеки пагинации

pip install fastapi-pagination

pip install --no-index --find-links=\\172.30.44.161\public fastapi-pagination

Использование пагинации

GET /category?page=1&size=10

- items — список элементов на текущей странице,
- total — общее количество элементов,
- page — текущая страница,
- size — размер страницы (число элементов),
- pages — общее количество страниц.

Request URL

```
http://127.0.0.1:8001/api/v1/category?page=1&size=3
```

Server response

Code	Details
200	Response body


```
{
  "items": [
    {
      "title_category": "Напитки",
      "id_category": 1
    },
    {
      "title_category": "Пиццы",
      "id_category": 2
    },
    {
      "title_category": "Десерты",
      "id_category": 3
    }
  ],
  "total": 3,
  "page": 1,
  "size": 3,
  "pages": 1
}
```

Вывод с пагинацией

```
from fastapi_pagination import Page
router = APIRouter()
@router.get("/endpoint", response_model=Page[ИмяКласса])
def имя_функции_запроса(session: Session = Depends(get_session)):
    return имя_функции(session)
```




```
from fastapi_pagination.ext.sqlmodel import paginate
def имя_функции(session: Session = Depends(get_session) , page: int = 1, size: int = 10):
    try:
        sql = select(ИмяКласса)
        return paginate(session , sql)
    except Exception as e:
        session.rollback()
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                            detail=f"Внутренняя ошибка сервера: {str(e)}")
```

Подключение пагинации при запуске сервера (файл main.py)

```
from fastapi import FastAPI
from contextlib import asynccontextmanager
@asynccontextmanager
async def on_startup(app: FastAPI):
    init_db()
    yield
    close_db()
app_v1 = FastAPI(lifespan=on_startup)
app_v1.include_router(router, prefix="/api/v1")
add_pagination(app_v1)
```

Валидация



FastAPI

Подключение библиотек валидации

pip install email-validator

pip install --no-index --find-links=\\172.30.44.161\public email-validator

Валидация

```
app/
├── schemas/
│   ├── schemas.py
```

В папке schemas находятся структуры данных, которые описывают формат входящих данных и исходящих ответов

Валидация при добавлении

```
from fastapi import APIRouter, Depends, HTTPException, status, Response
from app.models.имя_модуля import ИмяКласса  # ORM-модель (SQLModel)
from app.schemas import ИмяPydanticДляPOST    # Pydantic-модель для валидации входных данных

def имя_функции(    data: ИмяPydanticДляPOST,    session: Session = Depends(get_session)):
    """    Добавление новой записи    """
    try:
        obj = ИмяКласса(
                имяполя_1=data.имяполя_1,
                имяполя_2=data.имяполя_2
                )
        session.add(obj)
        session.commit()
        session.refresh(obj)
        return obj
    except IntegrityError:
        session.rollback()
        raise HTTPException(status_code=400, detail="Ошибка:  нарушение целостности данных")
    except Exception as e:
        session.rollback()
        raise HTTPException(status_code=500, detail=f"Внутренняя ошибка сервера: {str(e)}")
```

```
from pydantic import BaseModel
class ИмяPydanticДляPOST(BaseModel):
        имяполя_1: тип
        имяполя_2: тип
```

Валидация при добавлении

```
from fastapi import APIRouter, Depends, HTTPException, status, Response
from app.models.имя_модуля import ИмяКласса  # ORM-модель (SQLModel)
from app.schemas import ИмяPydanticДляPOST    # Pydantic-модель для валидации входных данных

def имя_функции(    data: ИмяPydanticДляPOST,    session: Session = Depends(get_session)):
    """    Добавление новой записи    """
    try:
        data_dict = data.dict()
        obj = ИмяКласса(**data.model_dump(exclude_unset=True)) # Создаем ORM объект из Pydantic-схемы
        session.add(obj)
        session.commit()
        session.refresh(obj)
        return obj
    except IntegrityError:
        session.rollback()
        raise HTTPException(status_code=400, detail="Ошибка:  нарушение целостности данных")
    except Exception as e:
        session.rollback()
        raise HTTPException(status_code=500, detail=f"Внутренняя ошибка сервера: {str(e)}")
```

```
from pydantic import BaseModel
class ИмяPydanticДляPOST(BaseModel):
        имяполя_1: тип
        имяполя_2: тип
```

Валидация при добавлении

```
from fastapi import Depends, HTTPException, status, Response
from app.models.имя_модуля import ИмяКласса  # ORM-модель (SQLModel)
from app.schemas import UserSchema    # Pydantic-модель для валидации входных данных
from fastapi import Form

def имя_функции(     data: UserSchema = Depends(UserSchema.as_form),    session: Session = Depends(get_session)):
    """Добавление новой записи с использованием данных формы"""
    try:
        obj = ИмяКласса(**data.model_dump(exclude_unset=True))
        session.add(obj)
        session.commit()
        session.refresh(obj)
        return obj
    except IntegrityError:
        session.rollback()
        raise HTTPException(status_code=400, detail="Ошибка")
    except Exception as e:
        session.rollback()
        raise HTTPException(status_code=500,
                detail=f"Внутренняя ошибка сервера: {str(e)}")
```

```
from pydantic import BaseModel
from fastapi import Form

class UserSchema(BaseModel):
    username: str
    password: str

    @classmethod
    def as_form(       cls,
        username: str = Form(..., description="Имя пользователя"),
        password: str = Form(..., description="Пароль")    ):
        return cls(username=username, password=password)
```

Валидация при редактировании (PATCH-запрос)

```
from fastapi import Depends, HTTPException, status
from app.db.session import get_session
from app.models.имя_модуля import ИмяКласса

def имя_функции(идентификатор: int, data: ИмяPydanticДляPATCH , session: Session = Depends(get_session)) -> ИмяКласса:
    """    Изменение    """
    try:
        result= session.get(ИмяКласса, идентификатор)
        if not result :
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"идентификатор не найден")
        for key, value in data.model_dump(exclude_unset=True).items():
            setattr(result, key, value)
        session.commit()
        session.refresh(result)
        return result
    except Exception as e:
        session.rollback()
        raise HTTPException( status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                                    detail=f"Внутренняя ошибка сервера: {str(e)}")
```

```
from pydantic import BaseModel
class ИмяPydanticДляPATCH (BaseModel):
        имяполя_1: тип
        имяполя_2: тип
```

Валидация входящих данных

```
rus_regex = r'^[А-Яа-яЁё\s-]+$'
class ИмяКласса(BaseModel):
    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "second_name": "Иванов",
                "first_name": "Иван",
                "patronymic_name": "Иванович",
                "email": "user@example.com",
                "password": "Password123!",
                "date_birth":"2004-10-15"
            }       }    )
    second_name: constr(pattern=rus_regex, max_length=50) | None = None
    first_name: constr(pattern=rus_regex, max_length=50) | None = None
    patronymic_name: constr(pattern=rus_regex, max_length=60) | None = None
    email: EmailStr
    password: str = Field(..., description="Пароль должен содержать латинские буквы, цифры и спецсимволы",min_length=8)
    date_birth: date | None = None

    @field_validator('password')
    def password_complexity(cls, v):
        # Регулярка для пароля (пример: должна содержать цифры, буквы, спецсимволы)
        pattern = r'^(?=.*[A-Za-z])(?=.*\d)(?=.*[!@#$%^&*()_+=-]).{8,}$'
        if not re.match(pattern, v):
            raise ValueError('Пароль должен содержать латинские буквы, цифры и спецсимволы, длина больше 8')
        return v
```

```
class Profile(SQLModel, table=True):
    id_users: int = Field(default=None, primary_key=True)
    second_name: Optional[str]= Field(max_length=50)
    first_name: Optional[str]= Field(max_length=50)
    patronymic_name: Optional[str]= Field(max_length=60)
    date_birth:Optional[date]
    role_id: int = Field(foreign_key="roles.id_role",default=1)
    email: str = Field(max_length=30, unique=True,default=None)
    password: str = Field(max_length=200,default=None)
    login_date: Optional[date] = Field(default_factory=date.today)
```

pip install email-validator

Аутентификация



FastAPI

Аутентификация: установка библиотек

pip install argon2-cffi passlib python-jose

Библиотека argon2-cffi

argon2-cffi - Библиотека для безопасного хеширования паролей
Passlib — это высокоуровневая библиотека для работы с разными алгоритмами хэширования паролей.
Passlib использует обертку над настоящей низкоуровневой библиотекой ` argon2-cffi `.
(Passlib может хэшировать и проверять пароли через алгоритм argon2-cffi)

Библиотека passlib

passlib - Библиотека для безопасного хеширования паролей

- Поддержка множества алгоритмов хеширования (bcrypt, argon2, pbkdf2 и др.)
- Обеспечивает простой интерфейс для хеширования и проверки паролей
- Рекомендуется использовать для безопасного хранения паролей в веб-приложениях

Библиотека python-jose

python-jose - Библиотека для работы с JWT (JSON Web Tokens)

- Поддержка создания, шифрования, валидации и декодирования JWT
- Поддерживает различные алгоритмы подписи (HS256, RS256 и др.)
- Позволяет удобно обрабатывать ошибки JWT через `JWTError` из `jose.exceptions`
- Часто используется для аутентификации и авторизации в веб-приложениях

Установка библиотек

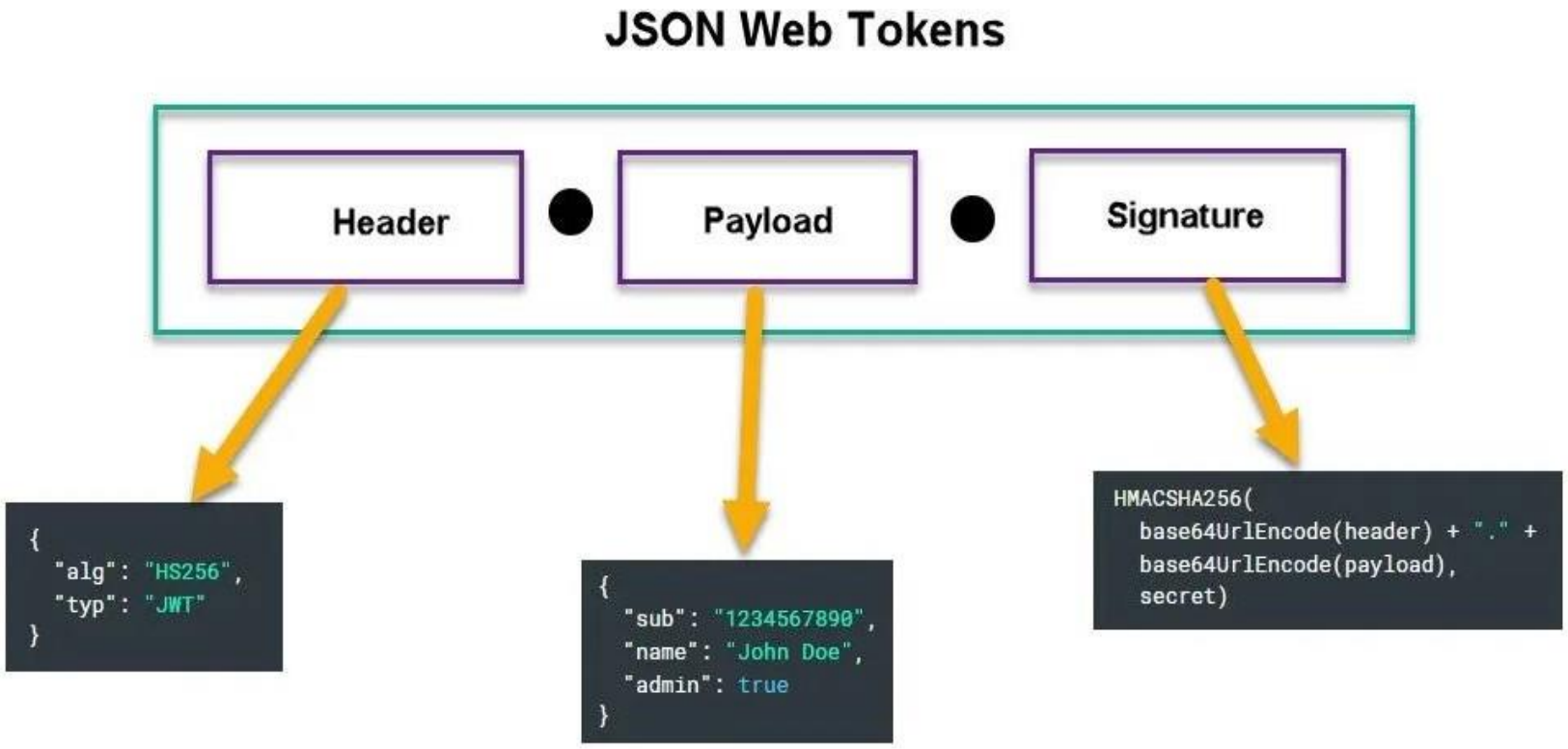
pip install argon2-cffi passlib python-jose

Установка с локального репозитория

pip install --no-index --find-links=\\172.30.44.161\public argon2-cffi
pip install --no-index --find-links=\\172.30.44.161\public passlib
pip install --no-index --find-links=\\172.30.44.161\public python-jose

Что такое JWT токен?

JWT (JSON Web Token) — это стандарт для создания токенов доступа, который широко используется в современных веб-приложениях и API для аутентификации и безопасного обмена данными.



Ключи (claims) полезной нагрузки

Стандартные ключи payload JWT:
iss (issuer) — издатель токена, кто его создал
sub (subject) — тема токена, обычно идентификатор пользователя
aud (audience) — аудитория, кому предназначен токен (например, сервис или клиент)
exp (expiration time) — время истечения срока действия токена (timestamp)
nbf (not before) — время, до которого токен ещё не действителен
iat (issued at) — время выпуска токена (timestamp)
jti (JWT ID) — уникальный идентификатор токена, используется для отзыва и отслеживания

пользовательские ключи, например:
user_id, username, email — информация о пользователе
roles, permissions — права и роли пользователя
type — тип токена (access, refresh и др.)
другие бизнес-данные, нужные приложению

Как работает JWT?

1. Пользователь входит в систему (логин/пароль или OAuth).
2. Сервер генерирует JWT и отправляет его клиенту.
3. Клиент сохраняет токен (в куках) и отправляет его в заголовке Authorization при каждом запросе
4. Сервер проверяет подпись и данные в токене. Если всё верно — разрешает доступ.

Алгоритмы JWT: HS256 и RS256

Характеристика	HS256	RS256
Тип алгоритма	Симметричный	Асимметричный
Ключ для подписи	Один секретный ключ	Приватный ключ
Ключ для проверки	Тот же секретный ключ	Публичный ключ
Безопасность	Менее безопасен, риск компрометации секрета	Более безопасен, приватный ключ не раскрывается
Производительность	Быстрее	Медленнее
Использование	Подходит для небольших систем	Рекомендуется для распределённых систем

Алгоритмы JWT: HS256 и RS256 используют разные типы криптографии

HS256 (HMAC + SHA-256)

- ✓ Симметричный алгоритм.
- ✓ Для подписи и проверки подписи используется один и тот же секретный ключ (shared secret).
- ✓ Сервер, который создает токен и сервер, который его проверяет, должны иметь один и тот же секрет.
- ✓ Проверка подписи — это повторное вычисление HMAC-сигнатуры с этим секретным ключом и сравнением с подписью из токена.
- ✓ Преимущества: быстрее и проще в реализации.
- ✓ Недостатки: все стороны, которые проверяют токен, должны доверять друг другу и хранить один секрет, что может создавать риски.

Алгоритмы JWT: HS256 и RS256 используют разные типы криптографии

RS256 (RSA + SHA-256)

- ✓ Асимметричный алгоритм.
- ✓ Используются пара закрытый ключ / открытый ключ.
- ✓ Токен подписывается приватным (закрытым) ключом.
- ✓ Проверка подписи делается с помощью соответствующего публичного (открытого) ключа.
- ✓ Публичный ключ может распространяться многим сервисам для проверки токена, приватный — хранится только на сервере подписания.
- ✓ Преимущества: выше безопасность, можно распределять публичный ключ без риска компрометации приватного, удобна ротация ключей.
- ✓ Недостатки: требует более сложной инфраструктуры (генерация ключей, хранение, распространение).

Ресурсы для тестирования JWT токенов

https://logto.io/ru/jwt-decoder

EdgeЦентр JWT Decoder

devmark.ru JWT Parser

Алгоритм JWT: HS256

```
from jose import jwt, JWTError
from fastapi import HTTPException, status

SECRET_KEY = "your_secret_key"
ALGORITHM = "HS256"

def verify_jwt_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")
        return username
    except JWTError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")
```

Методы для работы с токеном

- Для авторизации реализовать:
  - /api/v1/auth/login для получения токена.
  - /api/v1/auth/refresh для обновления токена.

Метод авторизации (login): отвечает за проверку учётных данных пользователя (логин/пароль), аутентификацию и выдачу пары токенов (Access и Refresh).

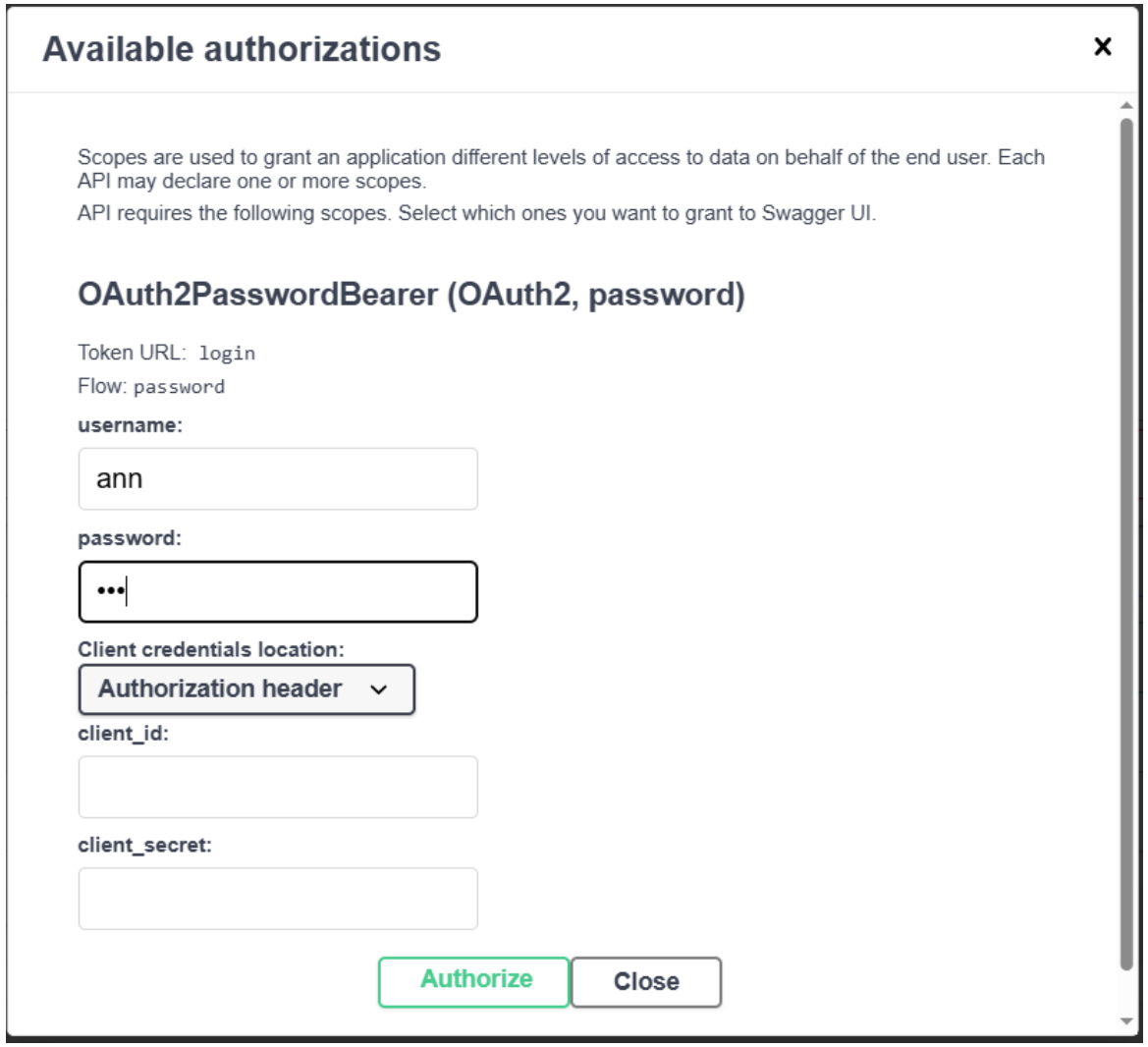
Метод получения/обновления токена (refresh): используется для продления срока действия сессии без повторного ввода пароля, выдаёт новый Access Token по валидному Refresh Token (выполняется обновления токена)

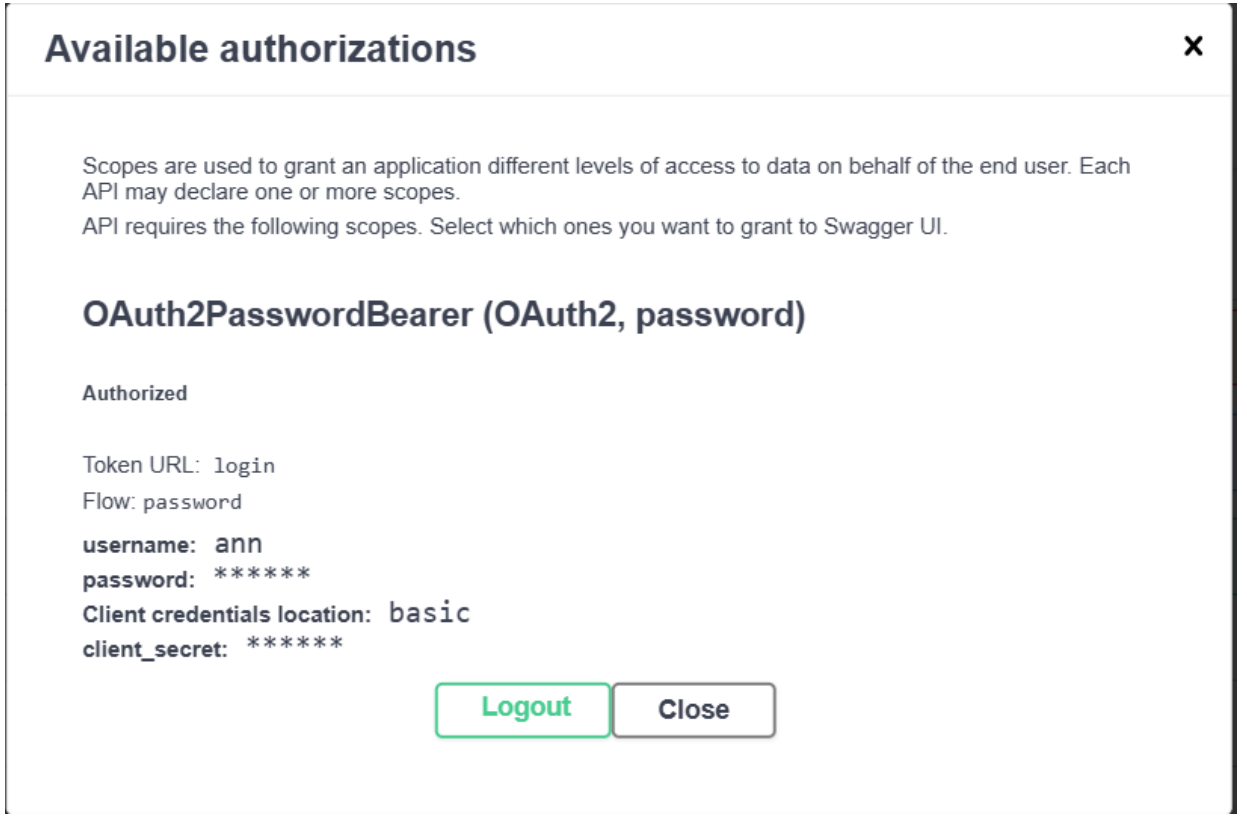
Методы для работы с токеном

Метод	**Функционал**
POST /api/v1/auth/login	Аутентификация, выдача Access и Refresh Token
POST /api/v1/auth/refresh	Обновление Access Token с помощью Refresh Token

Что такое OAuth2?

• **OAuth2 (Open Authorization)** — это открытый стандарт делегирования доступа, который позволяет приложениям получать ограниченный доступ к данным пользователя на другом сервисе (например, Google, GitHub) без передачи пароля.



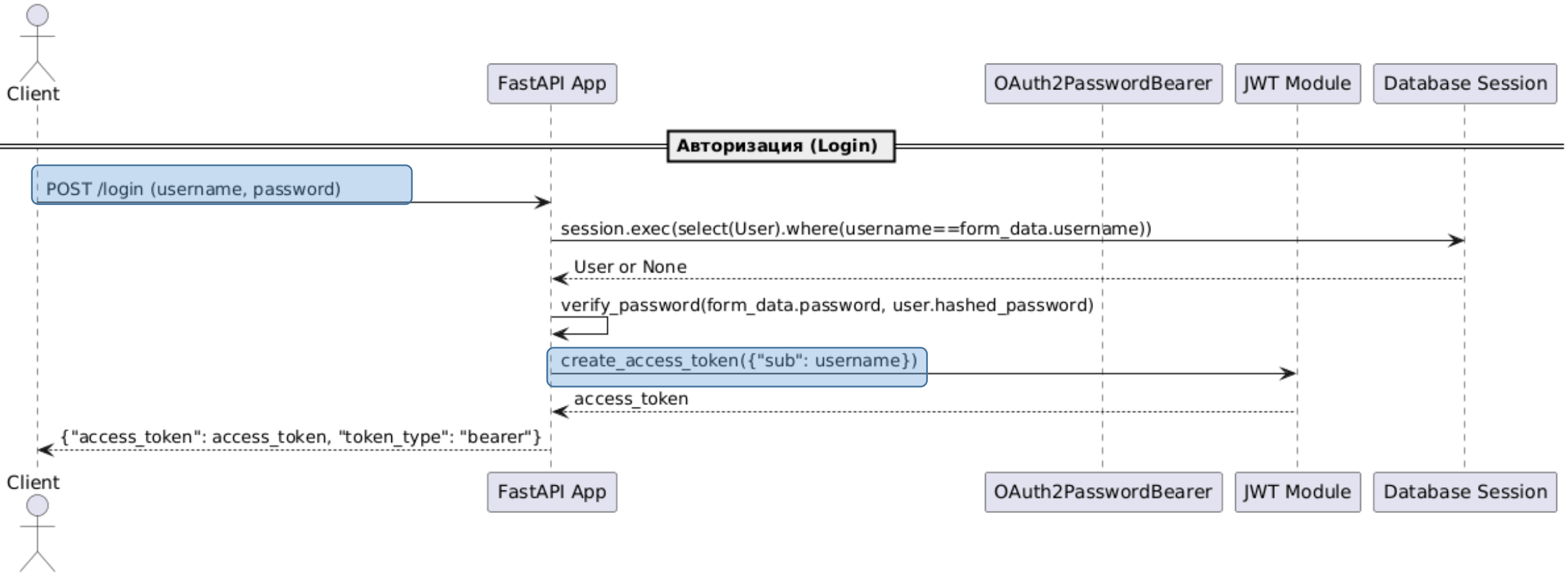


OAuth2PasswordBearer реализует логику проверки передачи токена в заголовке Authorization

Структура приложения

```
my_fastapi_project/
├── app/
│   ├── main.py                # Точка входа в приложение
│   ├── api/                   # Слой презентации (роутеры, эндпоинты)
│   │   ├── v1/
│   │   │   ├── user_api.py
│   │   │   └── ...
│   ├── models/                # Модели базы данных (слой доступа к данным)
│   │   ├── user.py
│   │   └── ...
│   ├── schemas/               # Схемы Pydantic для валидации данных
│   │   ├── user_schema.py
│   │   └── ...
│   ├── db/                    # Настройки базы данных и миграции
│   │   ├── database.py
│   │   └── session.py/
│   ├── core/                  # Настройки, конфигурации, безопасность
│      ├── config.py
│      └── security.py
├── tests/                     # Тесты, структурированные по слоям и модулям
│   ├── test_users.py
│   └── ...
├── .env                       # Переменные окружения
├── requirements.txt           # Зависимости
└── README.md                  # Документация проекта
```

Получение токена



Авторизация с выдачей токена

```
def login(form_data: UserSchema=Depends(UserSchema.as_form),    session: Session = Depends(get_session)):
        """     Авторизация      """
        user = session.exec(select(Users).where(Users.login == form_data.username)).one_or_none()
        if not user or not form_data.password== user.password:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Неверный логин или пароль")
        access_token = create_access_token(data={"sub": user.login})
        return {"access_token": access_token, "token_type": "bearer"}
```

```
from pydantic import BaseModel
from fastapi import Form

class UserSchema(BaseModel):
    username: str
    password: str

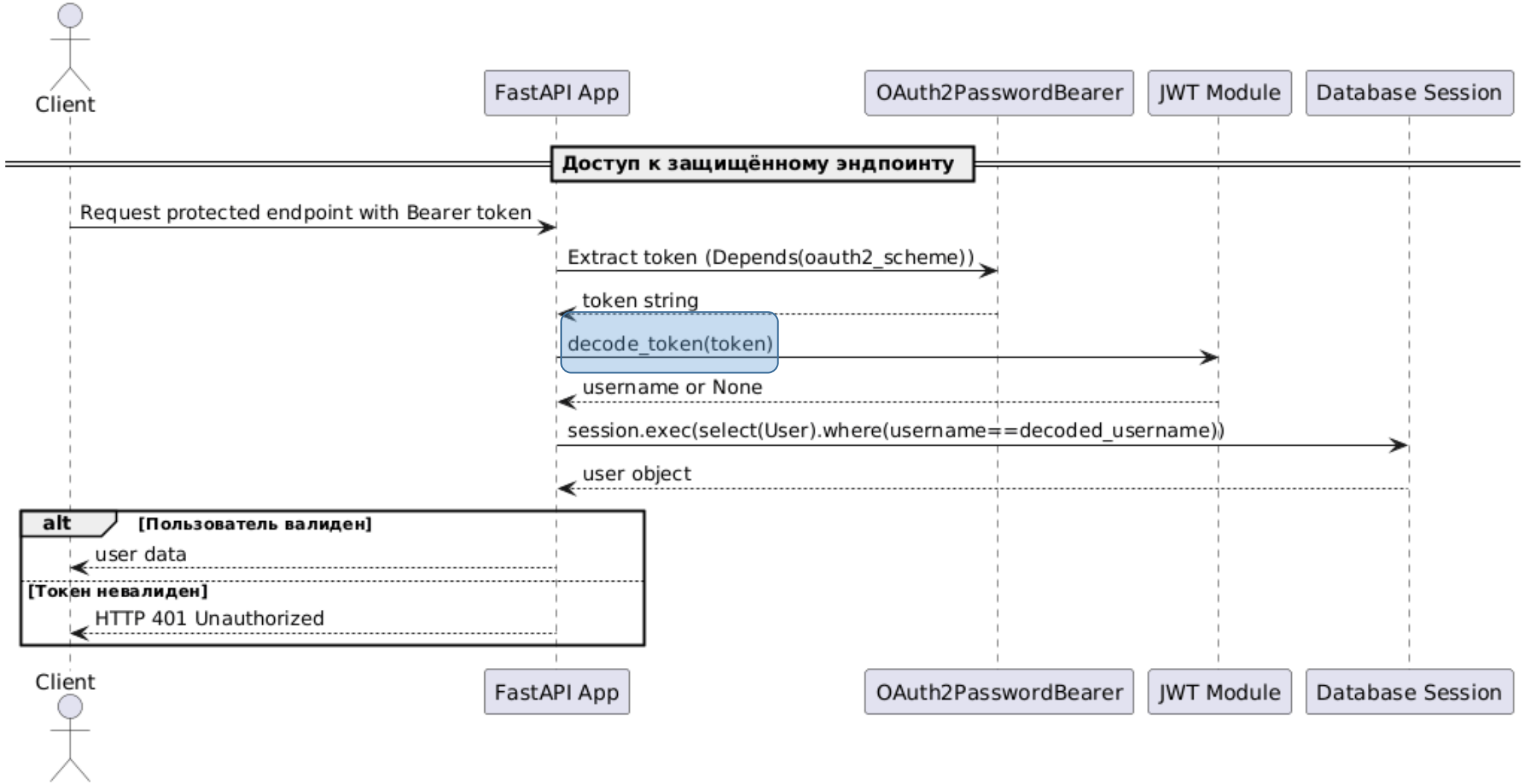
    @classmethod
    def as_form(        cls,
        username: str = Form(..., description="Имя пользователя"),
        password: str = Form(..., description="Пароль")    ):
        return cls(username=username, password=password)
```

Создание JWT access-токена

```
load_dotenv()  # загрузка переменных из .env
SECRET_KEY = os.getenv("SECRET_KEY")
ACCESS_TOKEN_EXPIRE_MINUTES=int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES"))
ALGORITHM=os.getenv("ALGORITHM")


def create_access_token(data:dict,expires_delta: timedelta=None):
    """    Создаёт JWT access-токен с данными и временем жизни
    :param data: словарь с данными для кодирования в токен (например, {"sub": username})
    :param expires_delta: timedelta - время действия токена
    :return: JWT строка
    """
    to_encode=data.copy()  #Копирует переданные данные
    # Добавление в них времени истечения срока действия (exp) — обязательное поле JWT
    expire=datetime.now()+(expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp":expire})
    #Возвращает закодированный JWT токен с использованием секретного ключа и алгоритма
    return jwt.encode(to_encode,SECRET_KEY, algorithm=ALGORITHM)
```

Получение объекта пользователя по токену



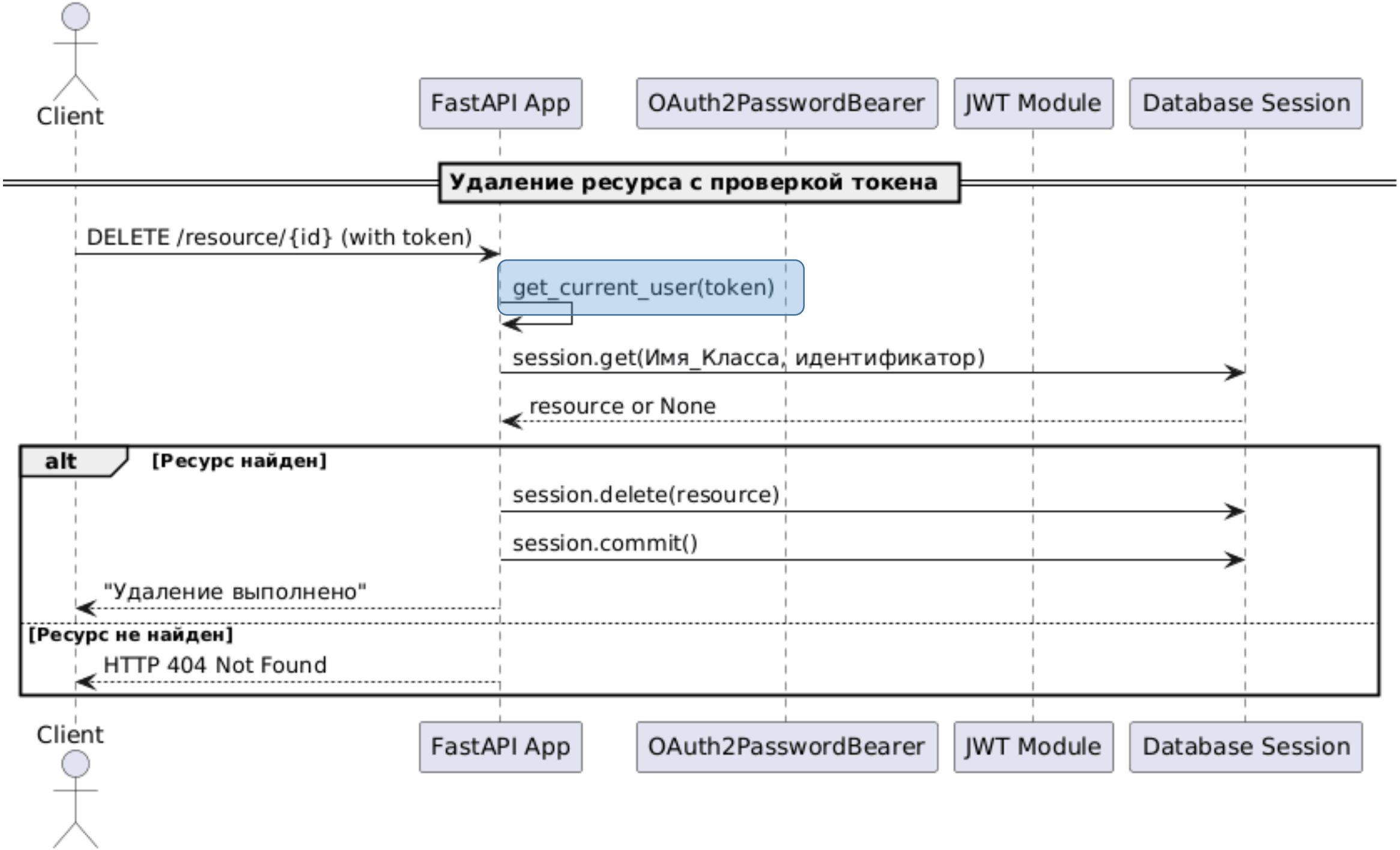
Декодирование JWT токена

Необходимо подключить библиотеку python-jose

```
from jose import jwt, JWTError

load_dotenv()  # загрузка переменных из .env
SECRET_KEY = os.getenv("SECRET_KEY")
ALGORITHM=os.getenv("ALGORITHM")

def decode_token(token:str):
    """
    Интроспекция JWT-токена (декодирование и проверка подписи) и возвращает имя пользователя
    :param token: JWT токен
    :return: полезная нагрузка
    """
    try:
        # Декодирование токена и проверка  подписи и срока её действия
        payload=jwt.decode(token,SECRET_KEY,algorithms=[ALGORITHM])
        return  payload.get("sub") #Извлекает полезную нагрузку (поле "sub" ,обычно username)
    except JWTError:
        return None
```


Client
FastAPI App
OAuth2PasswordBearer
JWT Module
Database Session
Удаление ресурса с проверкой токена
DELETE /resource/{id} (with token)
get_current_user(token)
session.get(Имя_Класса, идентификатор)
resource or None
alt
[Ресурс найден]
session.delete(resource)
session.commit()
"Удаление выполнено"
[Ресурс не найден]
HTTP 404 Not Found
Client
FastAPI App
OAuth2PasswordBearer
JWT Module
Database Session

Получение объекта пользователя по токену

```
from fastapi.security import OAuth2PasswordBearer # получение токена из запроса

# получения токена из заголовка Authorization Bearer
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def get_current_user(token:str=Depends(oauth2_scheme), session=Depends(get_session))-> ИмяКласса :
    """
    Получение объекта пользователя по токену
    :param token: токен из запроса (передаётся автоматически с помощью Depends и OAuth2PasswordBearer)
    :param session: сессия для базы данных (вызывается  при помощи Depends)
    :return: объект User или исключение HTTPException 401
    """
    username=decode_token(token)
    if not username:
        raise  HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,detail="Невалидный токен")
    user=session.exec(select(ИмяКласса).where(ИмяКласса.имя_поля==username)).first()
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,detail="Пользователь не найден")
    return user
```

Применение при реализации удаления (удалять может только авторизованный)

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlmodel import Session, select

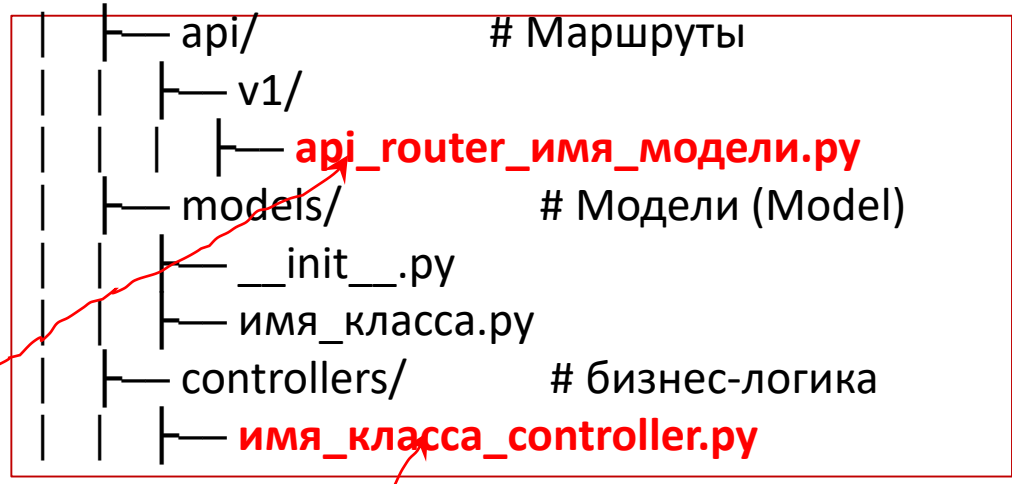
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

def имя_функции(идентификатор:  int, current_user: ИмяКласса = Depends(get_current_user),
                                        session: Session = Depends(get_session)):
    """     Удаление допустимо только при наличии токена     """

    result = session.get(Имя_Класса, идентификатор)
    if not result :
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="текст сообщения")
    session.delete(result)
    session.commit()
    return "Удаление выполнено"
```

Запрос на получение данных о текущем пользователе

```
@router.get("/me",response_model=Users)
def get_user_info_router(token: str=Depends(get_current_user)):
    """Получение данных о текущем пользователе"""
    return UserController.get_user_info(token)
```




```
class UserController:
    @staticmethod
    def get_user_info(user: Users = Depends(get_current_user), session=Depends(get_session)):
        """Получение данных о текущем пользователе"""
        if not user:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,  detail="Невалидный токен")
        return user
```

Метод обновления токена

```
def refresh_access_token(refresh_token: str):
    """Генерация нового токена"""
    username = decode_token(refresh_token)
    if not username:
        raise HTTPException(status_code=401, detail="невалидный токен")

    new_access_token = create_access_token(
        data={"sub": username, "type": "access"},
        expires_delta=timedelta(minutes=int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES"))),
    )
    return new_access_token
```

Контроль доступа на основе ролей (RBAC - Role-Based Access Control)



FastAPI

Доступ к функциям приложения

Актор	**Прецеденты (Use Cases)**
Неавторизованный	- Просмотр публичных данных (GET к публичным ресурсам)
	- Регистрация (POST /register)
	- Авторизация / Вход (POST /login)
Авторизованный	- Получить данные пользователя (GET /user/profile)
	- Обновить свои данные (PUT /user/profile)
	- Запрос защищённых данных (GET /protected/resource)
	- Запрос обновления токена (POST /auth/refresh)
Администратор	- Управление пользователями (GET/POST/PUT/DELETE /admin/users)
	- Управление содержимым (CRUD для администраторских ресурсов)

Как работает контроль доступа RBAC

1.При аутентификации пользователя определяется набор его ролей.
2.При каждой попытке доступа к ресурсу проверяется, есть ли у пользователя роль, которая разрешает соответствующее действие.
3.Доступ предоставляется или запрещается основываясь на проверке ролей.

Преимущества RBAC

- Централизованное управление доступом.
- Проще наладить соответствие политике безопасности.
- Легко масштабировать и изменять права без изменения кода пользователей.
- Повышает безопасность, разграничивая полномочия.

Проверка роли «администратор»

```
def admin_required(user: Users = Depends(get_current_user)):
    """
    Проверка роли администратора
    :param user: Объект класса Users
    """
    if not user.role=="admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Требуются права администратора"
        )
    return user
```

Работа с cookies



FastAPI

Cookie – что это?

Cookies в FastAPI — это базовая технология веба, позволяющая серверу сохранять небольшие данные на клиенте (браузере) для идентификации, авторизации, персонализации и аналитики.

Сервер устанавливает cookie в HTTP-ответе, а браузер автоматически отправляет их обратно при последующих запросах. Это ключевой механизм для сохранения состояния в stateless протоколе HTTP.

В FastAPI cookie применяются для:

- Авторизации пользователей через хранение токенов доступа в cookie;
- Поддержки работы корзин в интернет-магазинах, где cookie хранят идентификатор сессии пользователя;
- Сохранения пользовательских настроек интерфейса и предпочтений;
- Сбора аналитики и отслеживания поведения пользователей.

Основной пример установки cookie:

```
def set_cookie(response: Response):
    response.set_cookie(
        key="access_token",
        value="token123",
        max_age=3600,        # Время существования cookie в секундах: 1 час
        httponly=True,       # cookie недоступно для JavaScript, только для HTTP
                                протокола (для защиты от XSS))
        secure=True,         # Отправлять cookie только по HTTPS (для безопасности)
        samesite="lax"       # Политика SameSite (защита от CSRF)
    )
    return {"message": "Cookie успешно установлено"}
```

samesite — контролирует передачу cookie при кросс-доменных запросах

Strict —Cookie отправляются только если запрос инициирован с того же сайта (домена), которому принадлежат cookie. Если пользователь переходит на сайт или делает запрос с другого домена, cookie не отправится.

Lax —режим по умолчанию. Cookie отправляются при навигации с внешнего сайта (например, при переходе по ссылке), но не отправляются в случае кросс-доменных POST-запросов или AJAX.

None — отключает ограничения, cookie отправляются при любых запросах, в том числе кросс-доменных. При использовании SameSite=None обязательно добавить атрибут Secure, чтобы cookie передавался только по HTTPS, иначе браузеры отвергнут такой cookie.

```
from fastapi import Depends, HTTPException, Response, status
from passlib.hash import argon2
from app.schemas.user_schema import UserSchema

class UserController:

    @staticmethod
    def login(response: Response, data: UserSchema=Depends(UserSchema.as_form), session: Session = Depends(get_session)):
        """Авторизация"""
        user = session.exec(select(Users).where(Users.login == data.username)).one_or_none()
        if not user or not argon2.verify(form_data.password, user.password):
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Неверный логин или пароль")

        token = create_access_token(data={"sub": user.login, "role": user.role})
        response.set_cookie(
            key="access_token",
            value=token,
            httponly=True,
            secure=True,
            max_age=3600
        )
        return {"access_token": token, "token_type": "bearer"}
```

Выход из аккаунта

```
@router.post("/logout", summary='Выход из аккаунта')
def logout_user_router(response: Response):
    return UserController.logout_user(response)
```

```
class UserController:
    @staticmethod
        def logout_user(response: Response):
            # Удаляем cookie с токеном, например "access_token"
            response.delete_cookie(key="access_token")
            return {"message": "Пользователь успешно вышел из системы"}
```

Чтение cookie из запроса по имени

Допустим, что была создана cookie с именем «access_token», тогда для считывания значения cookie в функции необходимо указать имя параметра функции «access_token». Cookie(None) – задает значение None, если cookie с именем «access_token» не найден.

```
from fastapi import Depends, Response, status, Cookie
def имя_функции(access_token: Cookie(None)):
    return token
```

Асинхронное взаимодействие с базой данных



Библиотека asyncpg

asyncpg — высокопроизводительный асинхронный клиент для работы с базой данных PostgreSQL в Python. Она позволяет выполнять SQL-запросы и операции с базой данных эффективно, не блокируя главный поток выполнения, что особенно полезно для асинхронных приложений и сервисов с высокой нагрузкой.

Асинхронная работа: установка библиотек

pip install asyncpg

Изменение строки подключения (файл app/.env)

```
DATABASE_URL="postgresql+asyncpg://DB_USER:DB_PASSWORD@DB_HOST:DB_PORT/DB_NAME"
SECRET_KEY="scret_key_for_demo_new"
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

Подключение к БД при запуске сервера (файл app/main.py)

```
from fastapi import FastAPI
from contextlib import asynccontextmanager


@asynccontextmanager
async def on_startup(app: FastAPI):
    await init_db() # Асинхронная инициализация подключения к БД
    yield
    await close_db() # Асинхронное закрытие подключения


app_v1 = FastAPI(lifespan=on_startup)
```

Подключение к БД при запуске сервера (файл app/db/database.py)

```
import os
import asyncio
from dotenv import load_dotenv
from sqlalchemy.ext.asyncio import create_async_engine


load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")
# Создаем асинхронный движок с драйвером asyncpg
engine = create_async_engine(DATABASE_URL, echo=True, future=True)
```

Подключение к БД при запуске сервера (файл app/db/database.py)

```
from sqlmodel import SQLModel
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import sessionmaker

async_session = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(SQLModel.metadata.create_all)

async def close_db():
    await engine.dispose()  # Освобождение ресурсов
def get_engine():
    return engine
```

Подключение к БД при запуске сервера (файл app/db/session.py)

```
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.database import async_session

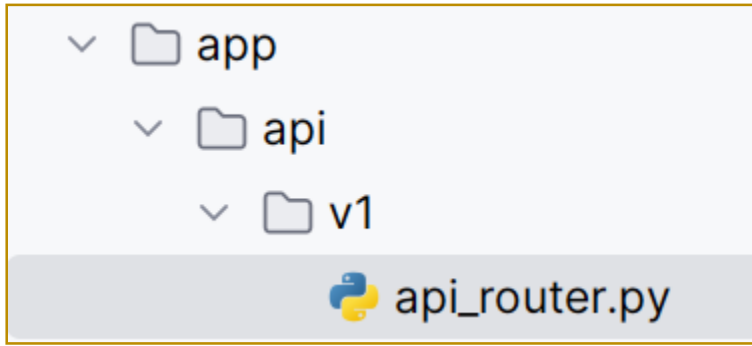

async def get_session() -> AsyncSession:
    async with async_session() as session:
        yield session
```

Создание APIRouter() – файл api_router.py

```
from fastapi import APIRouter, HTTPException, Depends
from sqlmodel importSession
from app.db.session import get_session
from app.models.имя_файла_класса import имя_класса

router = APIRouter()

@router.delete("/endpoint",summary="Описание",tags=["имя_класса"])
async def del_router(параметры, session: AsyncSession = Depends(get_session)):
        return  await имя_функции(параметры, session)
```



Вызов асинхронной функции через await

```
async  def имя_функции(параметры, session: AsyncSession = Depends(get_session)):
    """Функция вызывает другую фсинхронную функцию"""
    result = await имя_вызываемой_функции()


async  имя_вызываемой_функции():
    pass
```